Midterm concept review

CSC103 Programming Fundamentals / Lecture 17

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

In pairs, design a method that returns the larger of two numbers. Use it to find the largest of three values without arrays.

Define a two-argument maximum method.

Learning objectives

- Connect concepts from the first half of the course
- Trace expressions, branches and loops
- Explain the cause of a programming error

CLO-1 and CLO-2
Review scope
Expressions and assignment
Control-flow tracing
Strings and methods

Review scope

- This review supports the midterm slot in the supplied outline.
- Topics: source execution, types, control flow, strings and methods.
- These questions are practice material, not an official examination.

Example

```
Explain each result before running code.
State assumptions about valid inputs.
Use a trace when variables change.
```

Expressions and assignment

- An expression result depends on operand types.
- Assignment stores a value after the expression is evaluated.
- Parentheses change grouping but do not change operand types.

Example

```
double a = 9 / 2;
double b = 9 / 2.0;
// Explain both values.
```

Control-flow tracing

- Record the condition result before entering each branch or loop.
- Count the final failed while condition check.
- Trace continue and break as control transfers.

Example

```
int s = 0;
for (int i=1; i<5; i++) {
  if (i==2) continue;
  s += i;
}
```

Strings and methods

- String content comparison uses equals.
- substring uses an exclusive end index.
- Primitive parameters receive copies of values.

Example

```
"CSC103".substring(0,3)
"CS".equals("cs")
// Explain the type of each result.
```

An integrated answer needs more than syntax

- A method signature must match its intended inputs and result.
- The body must handle ties as well as strict comparisons.
- A short manually solved case should accompany the implementation.

Example

```
Maximum of 7, 7 and 3 is 7.
A correct solution must preserve ties.
```

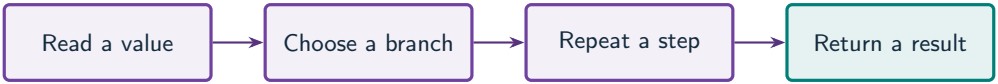
Nested calls can be traced in stages

- A returned result can become an argument to another call.
- Tracing the inner result first prevents confusion about execution order.
- This technique avoids duplicating the comparison logic.

Example

```
max(max(4,9),7)
Inner result: max(4,9) = 9
Outer result: max(9,7) = 9
```

Connect the first-half concepts



What to notice

Review by tracing how data moves through constructs, not by memorising syntax alone.

Key distinctions

Practice expression	Result	Reason
9 / 2	4	Both operands are int
9 / 2.0	4.5	One operand is double
"CSC103".substring(0,3)	CSC	End index is excluded
"CS".equals("cs")	false	Case-sensitive content comparison

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int total = 0;
for (int i = 1; i <= 4; i++) {
  total += i * 2;
}
System.out.println(total / 4.0);
```

First example: result

5.0
The accumulator values are 2, 6, 12 and 20.
Division by 4.0 produces a double result.

Control-flow tracing
Strings and methods

Guided practice

In pairs, design a method that returns the larger of two numbers. Use it to find the largest of three values without arrays.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Define a two-argument maximum method.
- Use its returned result as one argument to a second call.
- Test an ordinary case and a tie case.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution17 {
  public static void main(String[] args) throws Exception {
    int result = max(max(4, 9), 7);
    System.out.println(result);
    System.out.println(max(max(7, 7), 3));
  }
  static int max(int a, int b) {
    return a >= b ? a : b;
  }
}
```

Execution trace

Call	Arguments	Result
First inner call	4 and 9	9
First outer call	9 and 7	9
Tie inner call	7 and 7	7
Tie outer call	7 and 3	7

Follow the changing state in execution order.

Solution result

```
9
7
```

Why this result follows
Test an ordinary case and a tie case.

A common incorrect approach

```
int n=4;
while(n>0) {
System.out.println(n);
n++;
}
```

Example

The update moves away from the intended stopping boundary. Use `n--`. With Java `int` overflow the original loop eventually behaves differently, but it is still an incorrect countdown.

Boundary and failure checks

Check the stated input assumptions.
Create a one-page revision sheet with one traced example for each topic you find difficult.

Define max(a,b), then compute max(max(a,b),c). Test ascending values, descending values and ties.

Check your understanding

Explain one compile-time error, one runtime exception and one logic error from the first 16 lectures.

Write a prediction and explain the rule that supports it.

Answer and explanation

Examples: undeclared variable, parsing "x" as an int, and computing an average with unintended integer division.

The update moves away from the intended stopping boundary. Use n–. With Java int overflow the original loop eventually behaves differently, but it is still an incorrect countdown.

Compare cases and predict outcomes

Concept	Question to ask	Typical mistake
Expression	Which arithmetic type?	Integer division
Selection	Which condition matches first?	Incorrect branch order
Method	Where is the return stored?	Ignoring the returned value

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Without arrays, write a method that returns the larger of two values, then use it to find the largest of -3, -8 and -5.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static int max(int a, int b) {
  return a >= b ? a : b;
}
// In main:
System.out.println(max(max(-3, -8), -5));
```

- The inner call returns -3.
- The outer call compares -3 with -5 and returns -3.
- Initialising a largest value to zero would be wrong for this all-negative case.

A two-digit lottery decision

- Check exact order before reversed order, because the more specific award takes priority.
- Quotient and remainder separate the tens and units digits.
- State the two-digit input domain; this worked case fixes the lottery number for tracing.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: A two-digit lottery decision

Task

Use lottery 47 and guess 74. Award 10000 for exact order, 3000 for reversed digits, 1000 for one matching digit, otherwise 0.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture17 {
  public static void main(String[] args) throws Exception {
    int lottery = 47, guess = 74;
    int a = lottery / 10, b = lottery % 10;
    int c = guess / 10, d = guess % 10;
    int award;
    if (guess == lottery) award = 10000;
    else if (a == d && b == c) award = 3000;
    else if (a == c || a == d || b == c || b == d) award = 1000;
    else award = 0;
    System.out.println(award);
  }

}
```

Worked trace and expected result

Guess (lottery 47)	First matching rule	Award
47	Exact order	10000
74	Reversed order	3000
45 / 12	One match / none	1000 / 0

Expected console output

```
3000
```

Extend the ISB example

Practice

Why would checking one-digit matching before exact matching produce a wrong award?

Explain your reasoning

An exact match also satisfies the broad one-digit condition, so it would stop at the smaller award.

Lecture summary

- concepts from the first half of the course
- expressions, branches and loops
- the cause of a programming error

- Define a two-argument maximum method.
- Use its returned result as one argument to a second call.
- Test an ordinary case and a tie case.

After-class practice

Create a one-page revision sheet with one traced example for each topic you find difficult.

Syllabus reading
Lectures 1–16
Next lecture: Midterm practice workshop